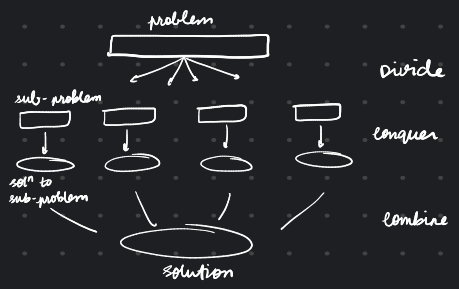
॥ श्रीगणेशाय नमः ॥

Divide and Conquer

It is a technique to divide a problem into smaller subproblems, solve them recursively, and then combine the solutions to get the solution to the original problem.



Given a function to compute on n inputs, divide-and-conquer suggests splitting the inputs into k distinct inputs, $1 < k \leq n$, yielding k sub-problems.
These sub-problems must be solved and a method must be found to combine the sub-solutions into a solution of the whole problem.
If the sub-problems are still relatively large, then divide-and-conquer can be applied again.

Dividing sub-problems into even smaller problems is done using recursion.
Division of a problem continues until the problem is small enough to be solved without splitting further.

```
ALGORITHM DivideAndConquer(P):
    if small(P) then return solution(P)

    divide P into smaller subproblems P1, P2, ..., Pk, k > 1

    for i = 1 to k do
        solution_i = DivideAndConquer(Pi)

    return combine(solution_1, solution_2, ..., solution_k)
```

Time complexity :

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small} \\ T(n_1) + \dots + T(n_k) + f(n) & \text{otherwise} \end{cases}$$

$g(n)$ → time for the small problem
$f(n)$ ← time to combine the sub-solutions.

It'll mostly be of the form $T(n) = \begin{cases} T(1) & n=1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n>1 \end{cases}$ s.t. a and b are constants and $n = b^k$.

E.g. for binary search, $T(n) = T(n/2) + c \Rightarrow T(n) = \log_2 n$

Binary search

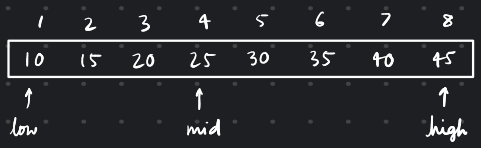
```
-- a is an array
-- i and l are the start and end indices of the search space
-- key is the value to search for
ALGORITHM BinarySearch(A, i, l, key):
    if l == i then return (A[i] == key) ? i : -1

    mid = floor((i + l) / 2)

    if key == A[mid] then return mid
    else if key < A[mid] then return BinarySearch(A, i, mid - 1, key)
    else return BinarySearch(A, mid + 1, l, key)
```

```
-- called
BinarySearch(A, 1, n, key)
```

Binary search finds elements in sorted arrays. Consider the following example.



To search for $x = 15$,

check the middle : $\left\lfloor \frac{low + high}{2} \right\rfloor = 4$

check the middle : $\left\lfloor \frac{low + high}{2} \right\rfloor = 4$ ← mid

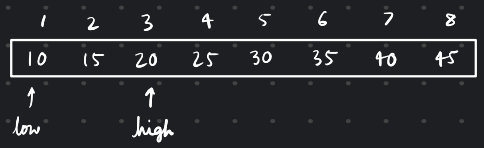
$arr[4] = 25$.

$15 < 25$.

We'll check the left sub-array.

The key, 15, is smaller than the middle element. So it can't be present in the right sub-array. Our search space reduces to half.

high = mid − 1



$mid = \left\lfloor \frac{low + high}{2} \right\rfloor = 2$

$arr[2] = 15$ ← required element.

Time complexity

$$T(n) = \begin{cases} c_1 & \text{if } n == 1 \\ T(n/2) + c & \text{if } n > 1 \end{cases}$$

$T(n) = O(\log n)$

space complexity for recursive binary search $= O(\log n)$

Using master theorem,

$f(n)$	$n^{\log_b a}$
c	1

$T(n) = \log n$

$T(n)$ — c

$T(n/2)$ — c

$T(n/2^k)$ — c

$T(n) = k \cdot c = O(\log n)$

Space $= k = O(\log n)$

$n/2^k = c \Rightarrow k = \log n$

The space complexity can be reduced using iterative binary search.

```
-- given an array A[1:n], search for key
ALGORITHM BinarySearch(A, n, key):
    low = 1, high = n

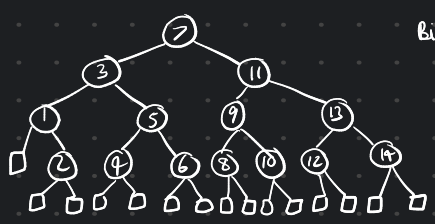
    while low <= high do
        mid = floor((low + high) / 2)

        if key == A[mid] then return mid
        else if key < A[mid] then high = mid - 1
        else low = mid + 1
    return -1
```

$T(n) = O(\lg n)$

Space $= O(1)$

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14



Binary decision tree for binary search.

Min-Max (finding the lowest and the highest elements of an array)

```
-- finding min and max of arr A[1:n] using brute force
ALGORITHM FindMinMax(A, n):
    min = max = A[1]

    for i = 2 to n do
        if A[i] < min then min = A[i]
        if A[i] > max then max = A[i]

    return min, max
```

A simple brute-force way would make 2(n-1), i.e. O(n) comparisons.

If an element > max, it won't be less than min. So we can reduce the # of comparisons.

```
for i = 2 to n do
    if A[i] < min then min = A[i]
    else if A[i] > max then max = A[i]
```

OR

```
for i = 2 to n do
    if A[i] > max then max = A[i]
    else if A[i] < min then min = A[i]
```

For this, best input will be s.t. next element > max so far.
e.g. 10, 12, 14, 23. (increasing order) — only 1st comparison will be made for every element.
(n-1) comparisons.

Worst case: decreasing order e.g. 23, 12, 3, 2, 1
(2(n-1)) comparisons. 2 comparisons for each element.

Avg case: A[i] > max ½ of the times.
½ times 1 comparison, ½ times 2 comparisons.

$$\frac{n-1}{2} \times 1 + \frac{n-1}{2} \times 2 = \frac{n-1+2n-2}{2} = \frac{3n-3}{2}$$

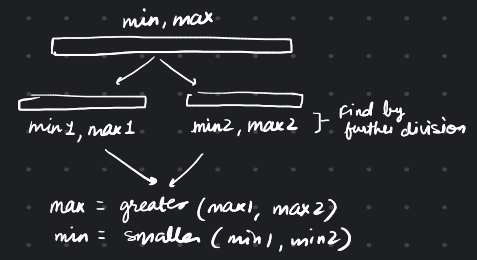
Best case: n-1 comparisons
Worst case: 2(n-1) comparisons.
Avg case: $\frac{3n}{2} - \frac{3}{2}$ comparisons

← O(n)

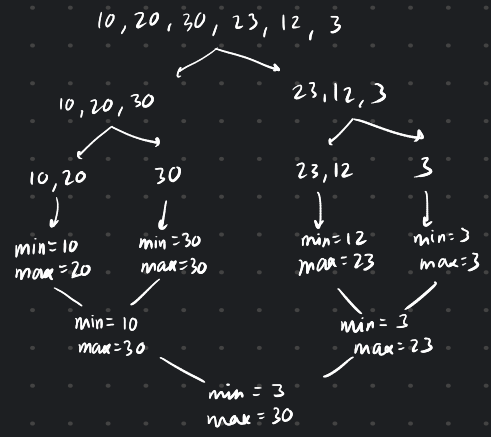
Divide and conquer method:

Divide the array into 2 halves and recursively find the min and max elements of the sub-arrays. Combine the soln's by comparing max and min values of both halves.

```
def find_min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]
    elif low == high - 1:
        return (arr[low], arr[high]) if arr[low] < arr[high] else (arr[high], arr[low])
    else:
        mid = (low + high) // 2
        min1, max1 = find_min_max(arr, low, mid)
        min2, max2 = find_min_max(arr, mid + 1, high)
        return min(min1, min2), max(max1, max2)
```




```
def find_min_max(arr, low, high, min, max):
    if low == high:
        min = max = arr[low]
    elif low == high - 1:
        if arr[low] < arr[high]:
            min = arr[low]
            max = arr[high]
        else:
            min = arr[high]
            max = arr[low]
    else:
        mid = (low + high) // 2
        min1, max1 = find_min_max(arr, low, mid, min, max)
        min2, max2 = find_min_max(arr, mid + 1, high, min, max)
        min = min1 if min1 < min2 else min2
        max = max1 if max1 > max2 else max2
```



of

$$C(n) = \begin{cases} C(n/2) + C(n/2) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$$

$C(n) = 2C(\frac{n}{2}) + 2$

$f(c)$ $n^{\log_b a}$

$2 < n$ ← greater by poly.

$O(n)$

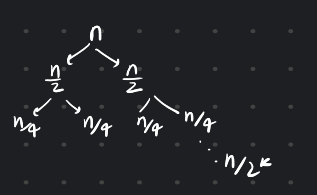
Ily, $T(n) = 2T(n/2) + C = O(n)$

Exact # of comparisons $= \frac{3n}{2} - 2$ [using substitution]

← Best, avg, and worst # of comparisons.
No method gives a smaller # of comparisons than Dal.

Disadvantage: Requires stack space.

$O(\log_2 n)$



$\frac{n}{2^k} = 1$

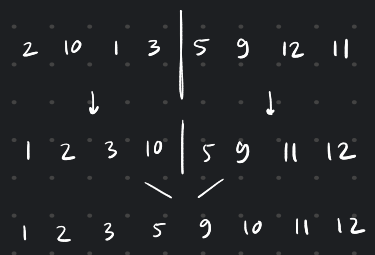
$k = \log_2 n$

space = $k = O(\log n)$

Merge Sort

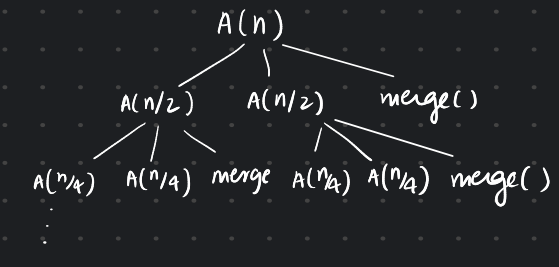
It sorts arrays by dividing them into halves and sorting the halves recursively.

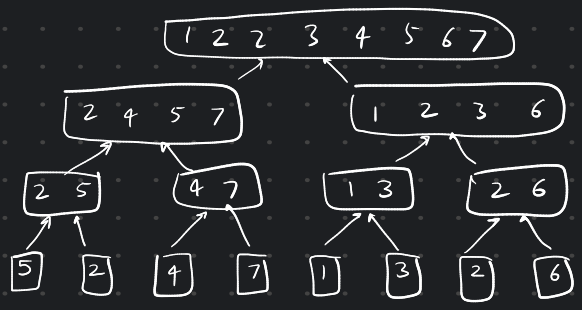
```
def merge_sort(A, p, r):
    if p < r:
        q = (p + r) // 2
        merge_sort(A, p, q)
        merge_sort(A, q+1, r)
        merge(A, p, q, r)
```



a size 1 array is already sorted.

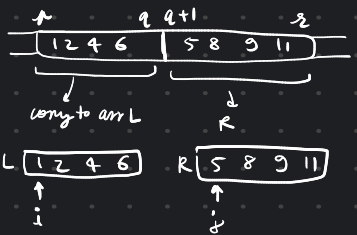
First we'll divide until we get an array of size 1.
Then we'll keep merging.



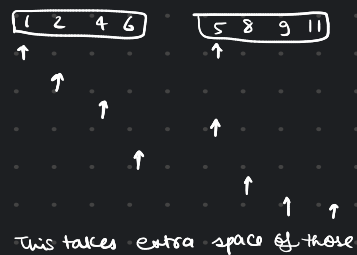


The main task is to merge two arrays.

Merging: we have sub-arrays A[p:q] and A[q+1: r] that are sorted.
We need to merge them into a sorted array.



Iterate through L and R.
Keep pushing the smaller ones into the original arr.



This takes extra space of those 2 arrays.

merge has 2 main parts:
- copying from A to L and R
- merging L and R to A.

Time to copy elements into L and R? $O(n)$.
For merging, 1 comparison per element. $O(n)$

So $T(\text{merge}) = O(n)$

Time complexity $= T(\frac{n}{2}) + T(\frac{n}{2}) + O(n)$

$$T(n) = 2T(\frac{n}{2}) + O(n)$$

```python
def merge(A, p, q, r):
    n1 = q - p + 1
    n2 = r - q
    L = [0] * (n1 + 1)
    R = [0] * (n2 + 1)

    # copy A[p..q] to L[0..n1-1]
    for i in range(n1):
        L[i] = A[p + i - 1]

    # copy A[q+1..r] to R[0..n2-1]
    for j in range(n2):
        R[j] = A[q + j]

    L[n1] = float('inf')
    R[n2] = float('inf')

    i = 0
    j = 0
    for k in range(p, r+1):
        if L[i] <= R[j]:
            A[k] = L[i]
            i = i + 1
        else:
            A[k] = R[j]
            j = j + 1
```

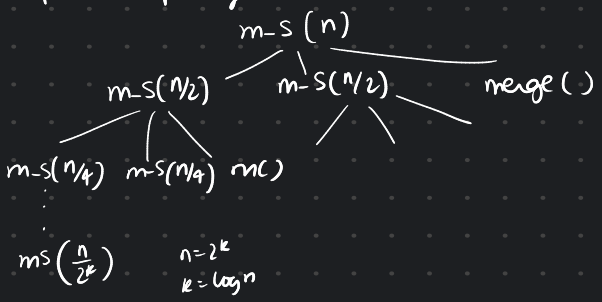
merge_sort(n) {
  merge_sort(n/2) → T(n/2)
  merge_sort(n/2) → T(n/2)
  merge(2 halves each n/2) → O(n)
}

$T(n) = 2 \cdot T(n/2) + O(n)$

f(n)	$n^{\log_b a}$
cn	n

$T(n) = O(n \log n)$

Space complexity



Also each time we use 2 size n arrays to copy the elements in the merge operations. $T(n) = O(n + \log n) = O(n)$

Space = O(n)

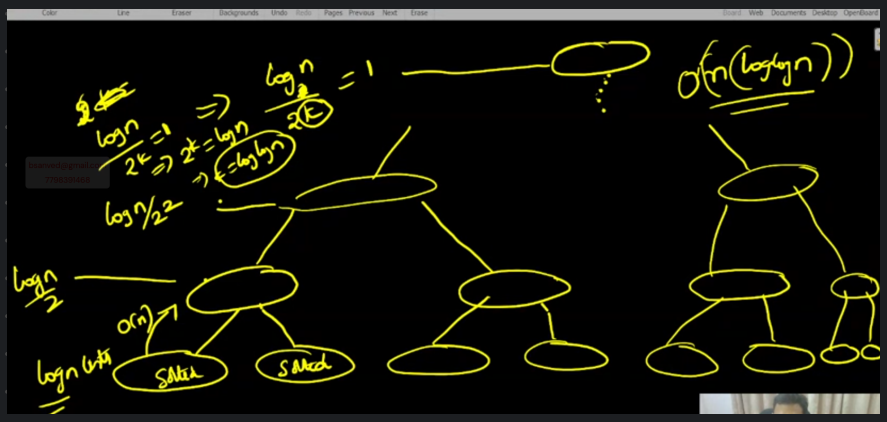
Q. Input = log n sorted sub-arrays, each of size $\frac{n}{\log n}$] Time complexity = ?

Output: One sorted array of n elements.



total # of elements $= \log n \cdot \frac{n}{\log n} = n$

Total n comparisons. O(n)



Q. Input : 'a' sorted subarrays, each of size b.
Output : single sorted array.

→



$\frac{a}{2^k} = 1$

$k = \log a$

$T(n) = ab \cdot \log_2 a$

Q. Given 2 sorted arrays, time complexity to find $A \cap B$?

→



Traverse i and j.
Increase the smaller till they are equal.
Then increase together till unequal.
Then increase smaller.
And so on.

$O(n)$

Time complexity:
Best case: $O(nlogn)$
Worst/average case: $O(nlogn)$

Extra space: $O(n)$

In-place: no
External: yes
Stable: yes

Quick sort



quick sort picks an element and puts it in its appropriate position ∋ elements to its left are smaller and those to the right are greater than it resp. (not necessarily sorted).

The left and right sub-arrays are sorted recursively.

Divide:
Partition the array A[p:r] into 2 (possibly empty) sub-arrays A[p:q-1] and A[q+1:r] such that each element of A[p:q-1] is less than or equal to A[q], which is in turn lesser than those in A[q+1:r].
Compute the index q as a part of the partitioning algorithm.

Conquer:
Sort the two sub-arrays A[p:q-1] and A[q+1:r] by recursively calling quick sort.

Combine:
Since the sub-arrays are sorted in-place, there's no need to combine them.

```
def quick_sort(A, p, r):
    if p < r:
        q = partition(A, p, r)
        quick_sort(A, p, q-1)
        quick_sort(A, q+1, r)
```

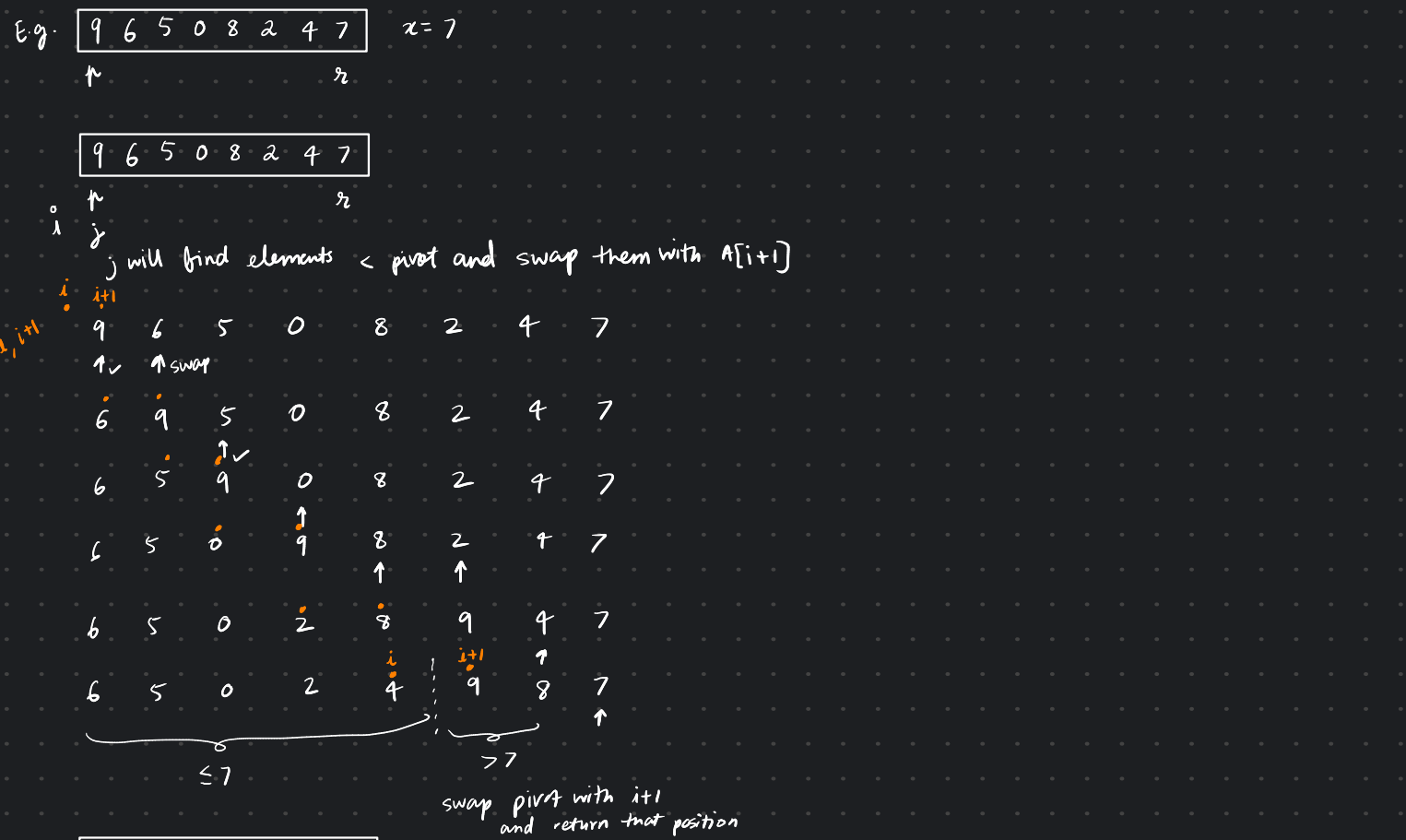
q is already sorted

```
def partition(A, p, r):
    x = A[r]
    i = p - 1
    for j in range(p, r):
        if A[j] <= x:
            i += 1
            A[i], A[j] = A[j], A[i]
    A[i+1], A[r] = A[r], A[i+1]
    return i+1
```

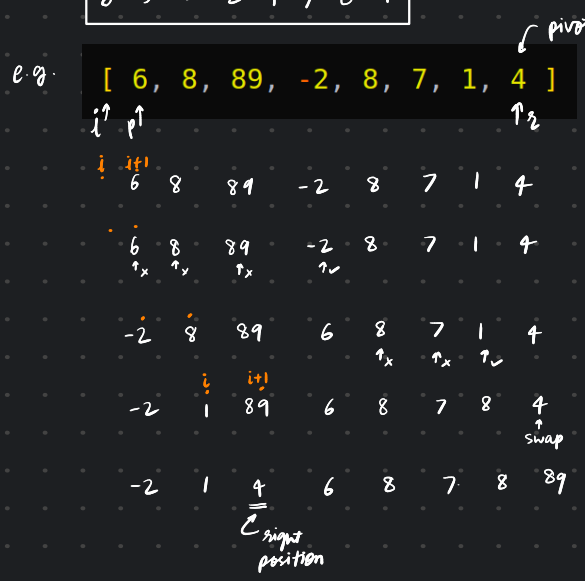
Time complexity:
- Best case: $O(nlogn)$
- Average case: $O(nlogn)$
- Worst case: $O(n^2)$

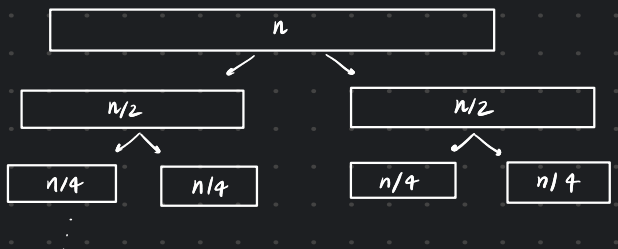
Extra space: $O(1)$

In-place: yes
Stable: no



6 5 0 2 4 7 8 9





Quick sort is really fast for small inputs.
For inputs < 20, quick sort is faster than merge sort.

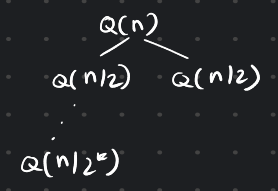
Best case:

The partitions are ≈ sized $n/2$.

Time complexity for partition: j runs from p to r-1.
$O(n)$

$T(n) = 2T(n/2) + cn$

$T(n) = O(n \lg n)$ space $= O(\lg n)$



$\frac{n}{2^k} = 1$

$k = \log_2 n$

Worst case:

QS calls partition on p to r and partition will return r.



$T(n) = T(0) + T(n-1) + O(n)$

$T(n) = T(n-1) + cn$

$= T(n-2) + 2n$

$= T(n-3) + 3n$

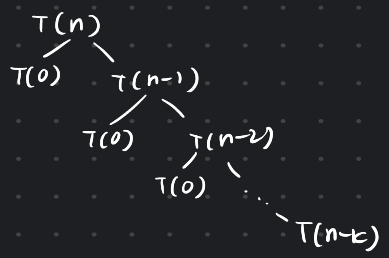
$= T(n-k) + kn$ $\quad n-k=0,\ k=n$

$= T(0) + n^2$

$= O(n^2)$

$T(n) = O(n^2)$

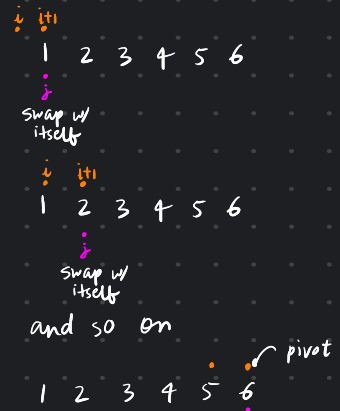
space $= O(n)$



$n-k=0$
$n=k$

What happens if quick sort is applied on a sorted array?

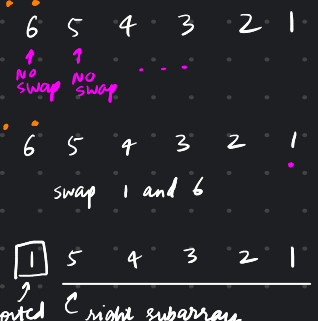
Increasing order:



$T(n) = T(0) + T(n-1)$

space $= O(n)$

Decreasing order:

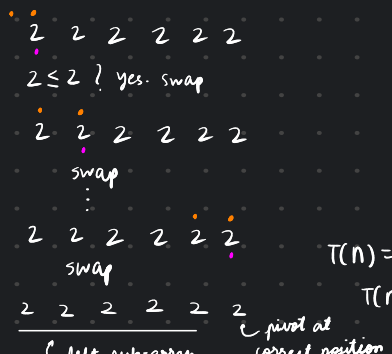


$T(n) = T(n-1) + O(n)$

$T(n) = O(n^2)$

space $(n) = O(n)$

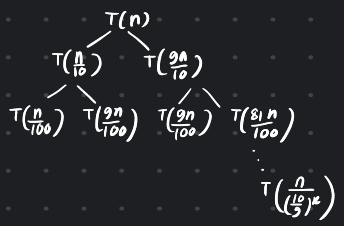
If all the elements are the same



$T(n) = T(n-1) + T(0) + cn$

$T(n) = O(n^2)$

Say each split is 1:9



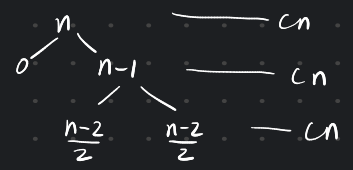
$\frac{n}{(10/9)^k} = 1$

$k = \log_{10/9} n$ $\quad T(n) = O(n \log n)$

Even if split is 1:99, we get $O(n \log n)$

So even if worst case $= O(n^2)$, quick sort is useful.

If the worst case happened once followed by the best case?



$$T(n) = cn + cn + cn + 2T\left(\frac{n-2}{2}\right)$$
$$\leq 3cn + 2T\left(\frac{n}{2}\right)$$

$O(n \log n)$

Quick sort PYQs

1.20.6 Quick Sort: GATE CSE 2006 | Question: 52

The median of n elements can be found in $O(n)$ time. Which one of the following is correct about the complexity of quick sort, in which median is selected as pivot?

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n^2)$
D. $\Theta(n^3)$

gatecse-2006 algorithms sorting easy quick-sort

median will approx. divide the array into sub-arrays of size 2

$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

$T(n) = O(n \log n)$

1.20.8 Quick Sort: GATE CSE 2009 | Question: 39

In quick-sort, for sorting n elements, the $(n/4)^{th}$ smallest element is selected as pivot using an $O(n)$ time algorithm. What is the worst case time complexity of the quick sort?

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n^2)$
D. $\Theta(n^2 \log n)$

gatecse-2009 algorithms sorting normal quick-sort

sub-arrays are divided as 1:3. $O(n \log n)$



1.20.9 Quick Sort: GATE CSE 2014 Set 1 | Question: 14

Let P be quicksort program to sort numbers in ascending order using the first element as the pivot. Let t_1 and t_2 be the number of comparisons made by P for the inputs $[1\ 2\ 3\ 4\ 5]$ and $[4\ 1\ 5\ 3\ 2]$ respectively. Which one of the following holds?

A. $t_1 = 5$
B. $t_1 < t_2$
C. $t_1 > t_2$
D. $t_1 = t_2$

gatecse-2014-set1 algorithms sorting easy quick-sort

first one is the worst-case.

1.20.7 Quick Sort: GATE CSE 2008 | Question: 43

Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let $T(n)$ be the number of comparisons required to sort n elements. Then

A. $T(n) \leq 2T(n/5) + n$
B. $T(n) \leq T(n/5) + T(4n/5) + n$
C. $T(n) \leq 2T(4n/5) + n$
D. $T(n) \leq 2T(n/2) + n$

gatecse-2008 algorithms sorting easy quick-sort

$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{4n}{5}\right) + O(n)$

$O(n \log_{\frac{5}{4}} n) = O(n \log n)$

Divide and conquer PYQs

Q1. Find the solution to the following recurrences equation

[GATE : 2MARKS]

$T(n) = T(n/2) + n$

$T(1) = 1$

$f(n) \quad n^{\log_b a}$

$n \gg 1$

$O(n)$

Q2. Assume that the last element of the set is used as partition element in Quicksort. If n distinct elements from the set [1 ... n] are to be sorted, give an input for which Quicksort takes maximum time.

[GATE : 2MARKS]

increasing / decreasing order

Q3. The recurrence relation that arises in relation with the complexity of binary search is:

[GATE : 2MARKS]

A. $T(n) = 2T\left(\frac{n}{2}\right) + k$, k is a constant

B. $T(n) = T\left(\frac{n}{2}\right) + k$, k is a constant

C. $T(n) = T\left(\frac{n}{2}\right) + \log n$

D. $T(n) = T\left(\frac{n}{2}\right) + n$

Q4. Merge sort uses:

[GATE : 1MARK]

a) Divide and conquer strategy
b) Backtracking approach
c) Heuristic search
d) Greedy approach

Q5.For merging two sorted lists of sizes m and n into a sorted list of size m + n, we require comparisons of

[GATE : 1MARK]

a) O(m)
b) O(n)
c) O(m + n)
d) O(logm + logn0)

Q6.The recurrence relation

T(1) = 2

T(n) = 3T (n/4) + n

has the solution T(n) equal to

a) O(n)
b) O(logn)
c) $O(n^{3/4})$
d) none of these

[GATE : 2MARK]

$f(n) \quad n^{\log_b a}$

$n > n^{\log_4 3} \approx n^{0.7924}$

differ by poly. $(n^{1-0.7924})$

Q7.Quicksort is run on two inputs shown below to sort in ascending order

(i) 1, 2, 3,....., n

(ii) n, n-1, n-2,....., 2, 1

Let C1 and C2 be the number of comparisons made for the inputs (i) and (ii) respectively. Then,

a) C1 < C2
b) C1 > C2
c) C1 = C2
d) We cannot say anything for arbitrary n

[GATE : 2MARKS]

Q8. Let T(n) be the function defined by $T(1) = 1,\ T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + \sqrt{n}$ for n>=2 [GATE : 2MARKS]
Which of the following statements is true?

A. $T(n) = O\sqrt{n}$
B. $T(n) = O(n)$
C. $T(n) = O(\log n)$
D. None of the above

$\sqrt{n} \ll n$

Q9.Let s be a sorted array of n integers. Let t(n) denote the time taken for the most efficient algorithm to determined if there are two elements with sum less than 1000 in s.Which of the following statement is true?
[GATE : 1MARK]

A. $T(n)$ is $O(1)$
B. $n \leq T(n) \leq n \log_2 n$
C. $n \log_2 n \leq T(n) < \frac{n}{2}$
D. $T(n) = \left(\frac{n}{2}\right)$

Q10.Randomized quicksort is an extension of quicksort where the pivot is chosen randomly. What is the worst case complexity of sorting n numbers using randomized quicksort? [GATE: 1MARK]

a) O(n)
b) O(n*log(n))
c) $O(n^2)$
d) O(n!)

Q11.Consider a list of recursive algorithms and a list of recurrence relations as shown below. Each recurrence relation corresponds to exactly one algorithm and is used to derive the time complexity of the algorithm.
[GATE : 2MARKS]

	Recursive Algorithm		Recurrence Relation
P	Binary search	I.	$T(n) = T(n-k) + T(k) + cn$
Q.	Merge sort	II.	$T(n) = 2T(n-1) + 1$
R.	Quick sort	III.	$T(n) = 2T(n/2) + cn$
S.	Tower of Hanoi	IV.	$T(n) = T(n/2) + 1$

Which of the following is the correct match between the algorithms and their recurrence relations?

A. P-II, Q-III, R-IV, S-I
B. P-IV, Q-III, R-I, S-II
C. P-III, Q-II, R-IV, S-I
D. P-IV, Q-II, R-I, S-III

Q13.Consider the Quicksort algorithm. Suppose there is a procedure for finding a pivot element which splits the list into two sub-lists each of which contains at least one-fifth of the elements. Let T(n) be the number of comparisons required to sort n elements. Then
[GATE : 2MARKS]

a) T(n) <= 2T(n/5) + n
b) T(n) <= T(n/5) + T(4n/5) + n
c) T(n) <= 2T(4n/5) + n
d) T(n) <= 2T(n/2) + n

Q14.In quick sort, for sorting n elements, the (n/4)th smallest element is selected as a pivot using an O(n) time algorithm. What is the worst-case time complexity of the quick sort?

$T(n) = T(\frac{n}{4}) + T(\frac{3n}{4}) + O(n)$

$O(n \log n)$

(A) θ(n)
(B) θ(n*log(n))
(C) θ(n²)
(D) θ(n² log n)
[GATE : 2MARKS]

Q15.Let P be a QuickSort Program to sort numbers in ascending order using the first element as pivot. Let t1 and t2 be the number of comparisons made by P for the inputs {1, 2, 3, 4, 5} and {4, 1, 5, 3, 2} respectively. Which one of the following holds?

a) t1 = 5
b) t1 < t2
c) t1 > t2
d) t1 = t2

[GATE : 1MARK]

Q16.The minimum number of comparisons required to find the minimum and the maximum of 100 numbers is _____.

[GATE : 2MARKS]

→ Using DaC, # of comparisons = $\frac{3n}{2} - 2 = 150 - 2 = 148$

An easier way to find it is by using **Tournament Method Technique** -

1. To find the smallest element in the array will take $\mathbf{n-1}$ **comparisons.**
2. To find the largest element -
   a. After the first round of Tournament , there will be exactly $\mathbf{n/2}$ **numbers** that will loose the round.
   b. So the biggest looser (the largest number) should be among these 50 loosers.To find the largest number will take $\mathbf{n/2-1}$.

Total Comparisons $= \mathbf{(n-1)+(n/2-1)=1.5n-2}$.

Q17.You have an array of n elements. Suppose you implement quick sort by always choosing the central element of the array as the pivot. Then the tightest upper bound for the worst case performance is

a) $O(n^2)$
b) O(n*log(n))
c) Theta(n*log(n))
d) $O(n^3)$

[GATE : 1MARK]

Q18. Which one of the following is the recurrence equation for the worst case time complexity of the Quicksort algorithm for sorting n(≥ 2) numbers? In the recurrence equations given in the options below, c is a constant.

a) T(n) = 2T (n/2) + cn
b) T(n) = T(n – 1) + T(1) + cn
c) T(n) = 2T (n – 1) + cn
a) T(n) = T(n/2) + cn

[GATE : 1MARK]

Q19. Assume that a mergesort algorithm in the worst case takes 30 seconds for an input of size 64. Which of the following most closely approximates the maximum input size of a problem that can be solved in 6 minutes?

[GATE: 2MARKS]

a) 256
b) 512
c) 1024
d) 2048

$T(n) = c \cdot n\log n$

$cn\log n = 30$ for $n = 64$

$c \cdot 64 \cdot \log 64 = 30$

$c = \frac{30}{64 \times 6} = \frac{5}{64}$

$c \cdot n\log n = 6 \text{ mins} = 6 \times 60 \text{ secs}$

$\frac{5}{64} \cdot n\log n = 6 \cdot 60$

$n\log n = 6 \cdot 12 \cdot 64 = 9 \cdot 2^9$

$= (\log_2 2^9)(2^9)$

$n = 2^9$